

Save Food

Project Report

Waste less food, feed more people.

Project	Save Food - a static anti-food-waste site with the EatMeFirst app
Author	_____
Course	_____
Date	16 June 2026

This report documents Save Food, a multi-page static website built to raise awareness of food waste and to help people act on it. The site pairs awareness content with EatMeFirst, a per-user fridge tracker, a donation flow, and a community pledge. It covers the problem the project addresses, the technology chosen to build it, and a reflection on the process.

1 Project overview

Save Food is a static, multi-page website about the global food-waste problem. It combines awareness content (a landing page, an About explainer, deep-dive story pages, a blog, and first-person Stories from around the world) with three ways for a visitor to take action: track their own food with the EatMeFirst app, donate toward feeding the hungry, and add their name to a Join Us pledge.

EatMeFirst is the centerpiece. It is a per-user fridge tracker that lets a person add groceries by photo, by barcode, or by quick-pick; it warns when items are about to expire, suggests recipes to use them up, and keeps a running tally of food and money saved versus wasted. The whole project runs as plain HTML, CSS, and JavaScript with no backend server and no build step, served locally with a single command.

2 Problem justification

Roughly a third of all food produced for human consumption is never eaten, while hundreds of millions of people go hungry. The waste is not only a moral problem but an environmental one: food that rots in landfill is a significant source of greenhouse-gas emissions, and every wasted meal also wastes the water, land, and labour that produced it. A large share of this loss happens at the end of the chain, in ordinary households, where food is bought, forgotten, and thrown away.

That last point is what motivated the project. The stage of the problem where an individual has the most direct leverage is their own kitchen. Save Food therefore does two things at once: it explains why food waste matters, so the visitor cares, and it puts a practical tool in their hands so they can act. EatMeFirst targets the specific moment waste occurs, when food is bought and then forgotten, by making what is in the fridge visible and by nudging the user to cook the items that will spoil first. The donation page and the Join Us pledge extend that single act of caring into supporting others and into a public commitment.

Why a static, backendless build

A deliberate constraint was to build the entire site as static files with no server of our own. For a learning project this keeps the scope honest and the cost at zero: the site can be served by a single command and deployed to any static host. Just as importantly, it pushes every sensitive concern out to a specialist service rather than asking a beginner project to secure it. Payments go to Stripe-hosted pages, so no card data and no secret key ever touch this code and the project stays out of PCI scope. User accounts and shared data use the Firebase client SDK, so there is no password store or database server to run. The EatMeFirst intelligence calls the Claude API directly from the browser with a key the user supplies and that stays on their

device. Choosing services over a custom backend was the decision that made a functional, genuinely useful site achievable within the project's scope.

3 Technology stack

The site is built on a Bootstrap template, re-skinned to a green Save Food identity, with all dynamic behaviour added in plain JavaScript. The table below summarises each layer; the notes that follow explain the main choices.

Layer	Technology and role
Markup and style	HTML5, CSS3, Bootstrap (grid and components), custom theme stylesheets (food, auth, eatmefirst, stories). No framework or bundler.
Interactivity	Vanilla JavaScript and ES modules. jQuery, WOW.js, Isotope, and imagesLoaded power the template's animations and layout.
Auth and data	Firebase Authentication (email/password, Google, GitHub) and Realtime Database for fridge sync, pledges, and per-post comments.
AI features	Anthropic Claude API for EatMeFirst photo recognition and recipe suggestions (default model claude-opus-4-8), called from the browser.
Barcode lookup	Open Food Facts API plus the browser BarcodeDetector for an optional camera scan.
Payments	Stripe Payment Links in test mode; the site links out and never handles card data.
Tooling	Python's http.server for local hosting; icon sets (Font Awesome, Ionicons, Material). No build pipeline.

Notes on key choices

- Firebase was chosen because it provides authentication and a real-time, multi-user database entirely from the client, which is exactly what a backendless site needs. The same exports power the login pages, the fridge sync, the comment threads, and the Join Us counter.
- Stripe Payment Links give a real, working payment flow (card entry, validation, declines, receipts) without a backend and without ever exposing the project to raw card data. Test mode keeps the demo honest: the flow is genuine but no money moves.
- The Claude API supplies the EatMeFirst smarts (reading a photo of groceries into items, and suggesting recipes). The key is entered in the app and stored only in the browser, so no secret is committed to the repository.
- Guests are not locked out: when no one is logged in, the fridge data falls back to the browser's localStorage, so the tool is usable immediately.

4 Reflection

Building Save Food taught me more about working within constraints than about any single technology. Committing to a static, backendless site early on forced me to think in terms of which trusted service owns each hard problem, rather than reaching for a server by default. That shift, from writing every feature myself to composing hosted services such as Firebase, Stripe, and the Claude API, is the lesson I expect to carry forward the most.

The work that went smoothly was the content and the visual identity. Re-skinning the Bootstrap template into a consistent green theme across many pages, and writing the Stories and blog content, was straightforward once the shared structure was in place. EatMeFirst was the most rewarding part to build because it turns the project's message into something a person can actually use.

What was challenging

- Several bugs came from the gap between the static files and the browser's rules. Social sign-in popups simply do not work from the `file://` protocol, so the site has to be opened through `http://localhost`. Tracking that down, and documenting it clearly, took longer than the feature itself.
- The password show/hide toggle was broken by three small faults at once: dead script paths, a mismatched element id, and an input that defaulted to visible. It was a good reminder to check assumptions one at a time instead of guessing.
- Keeping the blog comments from mixing meant keying each thread per post rather than sharing one global thread, a subtlety that is easy to miss until two articles show the same comments.
- Working without a backend was freeing but it also exposes data. Pledge names and emails are written straight from the browser, and tightening who can read that raw data is a Firebase security-rules task I have flagged but not yet done.

What I would do next

- Tighten the Firebase security rules so the stored pledge and comment data cannot be read openly.
- Add a small backend or serverless function so the Claude API call and any secrets live off the client, and so donation amounts could be passed securely.
- Introduce automated tests; right now every change is verified by hand in the browser, which does not scale as the site grows.
- Do an accessibility and performance pass: image sizes, colour contrast on the green theme, and keyboard navigation.

5 Conclusion

Save Food set out to do two things: explain why food waste matters and give people a concrete way to waste less. By pairing awareness content with the EatMeFirst tool, a working donation flow, and a community pledge, the project meets that goal within the limits of a static, client-only build. The remaining work, mostly around data security and testing, is well understood and would be the natural next step if the project continued.